EE8350 - AES Decode

Joel Kuehne <kuehn348@umn.edu>

This report details the verification plan and preliminary test framework for the FPGA AES Decryption Core from OpenCores. This core contains an implementation of the AES decryption algorithm described in FIPS-197 and is parameterizable for key lengths of 128, 192, and 256 bits. The core is verified against a behavioral reference model of the AES core and against selected test vectors from FIPS-197, NIST SP 800-38A, and AESAVS specifications. Primary coverage is obtained from system-level tests; additional coverage can be gained from module-level tests targeting the Decryption Engine and Key Expander individually as needed.

Introduction

The Advanced Encryption Standard (AES) is a symmetric key encryption/decryption standard based on the Rijndael block cipher algorithm. The OpenCores AES Decryption Core for FPGA implements the decryption portion of this algorithm. Described in "AES Decryption Core for FPGA: Specification", it supports compile-time configurations for 128, 192, and 256-bit decryption. It is designed for high throughput on FPGA targets and can achieve a peak throughput of 3Gbps for 256-bit keys while only requiring around 2000 LUTs. It is a fully synchronous design with one clock domain.

The core is split up into three components. The Key Expander accepts key text on the "kt" interface and expands it into a series of round keys. The Decryption Engine accepts input ciphertext on the "ct" interface and decrypts it into plaintext, which is returned on the "pt" interface. The Key Expander and Decryption Engine communicate through the Key Schedule Buffer, which implements a Last-In-First-Out (LIFO) buffer that holds the different round keys.

Top Module

OpenCores provides three different top modules for their AES core, one for each supported key length. The three different parameterizations are identical apart from the fact that they require different key expansion models and that they use different numbers of rounds as specified in the AES standard (10, 12, and 14 respectively for the 128, 192, and 256-bit configurations). The top-level module is connected directly to the testbench via the "pt", "ct", and "kt" interfaces as described in Table 1.

Decryption Engine

The decryption engine implements the four inverse operations required to decrypt the provided ciphertext. The inverses of AddRoundKey, MixColumns, ShiftRows, and SubBytes are implemented in InvAddRoundKey, InvMixColumns, InvShiftRows, and InvSubBytes

respectively. InvAddRoundKey reverses the Galois field addition by XORing the ciphertext with the round key. InvMixColumns reverses the column-mixing matrix operation by multiplying by the inverse matrix in the Galois field. InvShiftRows implements the inverse row-shift operation by repacking bytes. InvSubBytes computes the byte substitution via a pre-computed 8-input lookup table.

The engine accepts input ciphertext on the "ct" interface and stores it in the State register. The engine performs one decryption round every cycle. Taking into account the additional cycle of latency to assign the State register, each 128-bit decryption takes ROUND_COUNT + 1 cycles (11, 13, and 15 for 128, 192, and 256-bit key length configurations, respectively). The testbench drives ciphertext to the decryption engine from the "ct" interface, observes the primary output on the "pt" interface, observes the round key input for scoreboarding, and observes the input to the InvSubBytes module for coverage tracking.

Key Expander

The core implements three different key expansion modules, one for each supported key length. Each module implements the rotation, word substitution, and round constant addition as specified in FIPS-197. The substitution is implemented with a pre-computed 8-input lookup table. The result of the key expansion is sent to the key schedule buffer. Similar to the Decryption Engine, the Key Expander requires one cycle per round key plus an additional cycle of latency to populate the round key register (11, 13, and 15 cycles for 128, 192, and 256-bit key length configurations). The testbench drives key text to the expander on the "kt" interface and observes the expanded key output immediately before the Key Schedule Buffer.

Signal		Source	Width	Description
Ports	**clk**	TB	1	Core clock
	rst	TB	1	Active high synchronous reset
	kt	TB	128/192/256	Key input
	kt_vld	TB	1	Key input valid signal
	kt_rdy	DUT	1	Key input ready signal
	ct	TB	128	Ciphertext input
	ct_vld	TB	1	Ciphertext input valid signal
	ct_rdy	DUT	1	Ciphertext input ready signal
	pt	DUT	128	Plaintext output
	pt_vld	DUT	1	Plaintext output valid signal
Decrypt	**rkey_decrypt**	DUT	128	Decrypter key schedule buffer data
	rkey_vld_decrypt	DUT	1	Decrypter key schedule buffer valid
	next_rkey_decrypt	DUT	1	Decrypter key schedule request
	inv_shiftrows_out	DUT	128	Input to InvSubBytes module
Expand	**rkey_keyexp**	DUT	128	Expander key schedule buffer data
	rkey_vld_keyexp	DUT	1	Expander key schedule buffer valid
	rot_word_out	DUT	32	Input to word substitution LUT

Table 1: DUT connections for the AES testbench.

Verification Plan

Functional Methods

The primary goal of the verification of the decryption core is to ensure that the core correctly implements the AES decryption algorithm described in FIPS-197. Apart from this, the core must correctly implement ready/valid interfaces and must respond properly to reset assertion. Primary coverage is driven through constrained random verification at the top-level. Additional directed testing uses known test vectors from the FIPS-197, NIST SP 800-38A, and AESAVS specifications. These vectors were provided by OpenCores with the original repository source (because the test vector repository is very large, the raw values are omitted here). The complete verification plan is below.

ID	Requirement	Description
	Verification	
1	**Configuration**	
1.1	128/192/256-bit Key Length	Verify that the DUT supports compile-time configuration for 128, 192, and 256-bit key lengths.
	Run each test on all three DUT configurations.	
2	**Reset**	
2.1	Synchronous Warm Reset	Verify that DUT state transitions to 0 on reset and any stored key or ciphertext is dropped.
	Assert that the expander and decoder modules return to state 0 after reset.	
	Verify that transactions before and after reset are processed successfully.	
	Cover all round counter FSM states at the moment of reset assertion through CRV (11, 13, and 15 for 128, 192, and 256-bit parameterizations, respectively).	
3	**Decryption Engine**	
3.1	Plaintext X-Propagation	Verify that output signals on the “pt” interface are known when “pt_vld” is asserted.
	Assert that “pt” is known when “pt_vld” is asserted.	

3.2	Decryption Correctness	Verify that the Decryption Engine correctly implements the AES decryption algorithm.
	Verify that the plaintext output is consistent with that of the behavioral model.	
	Verify correct plaintext output against known test vectors from the FIPS-197, NIST SP 800-38A, and AESAVS specifications.	
3.3	InvSubBytes Correctness	Verify that the InvSubBytes module correctly implements the inverse S-box.
	Coverpoint on each byte lane of inv_shiftrows_out with 256 explicit value bins to cover all LUT entries.	
4	**Key Schedule Buffer**	
4.1	Round Key X-Propagation	Verify that output signals on the “rkey_decrypt” interface are known when “rkey_vld_decrypt” is asserted.
	Assert that “rkey_decrypt” is known when “rkey_vld_decrypt” is asserted.	
4.2	Round Key LIFO Ordering	Verify that round keys are returned on the “rkey_decrypt” interface in the opposite order in which they are provided on the “rkey_keyexp” interface.
	Assert LIFO ordering between the “rkey_decrypt” and “rkey_keyexp” interfaces.	
5	**Key Expander**	
5.1	Round Key X-Propagation	Verify that output signals on the “rkey_keyexp” interface are known when “rkey_vld_keyexp” is asserted.
	Assert that “rkey_keyexp” is known when “rkey_vld_keyexp” is asserted.	
5.2	Key Expander Correctness	Verify that the Key Expander correctly implements the AES key expansion algorithm.
	Verify that the round key output is consistent with that of the behavioral model.	
	Verify correct plaintext output against known test vectors from the FIPS-197, NIST SP 800-38A, and AESAVS specifications.	
5.3	Substitution Correctness	Verify that the word substitution module correctly implements the S-box transform.
	Verify that the round key output is consistent with that of the behavioral model.	
	Coverpoint on each byte lane of rot_word_out with 256 explicit value bins to cover all LUT entries.	

Table 2: Verification plan for the AES Decryption Core

Constrained random verification is the primary testing method used in the design. Key coverage items, including the hard-coded S-box LUT, can all be exercised using constrained random verification. Because this verification approach relies heavily on a non-trivial software model, our results are cross-checked against publicly available known test vectors. Constrained random verification is done in the testbench framework in Figure 1 below.

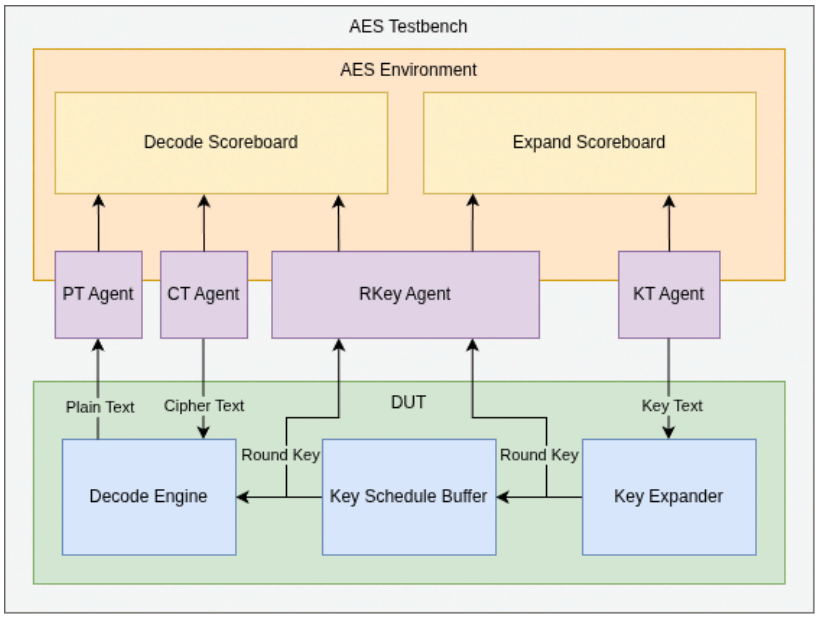


Figure 1: Primary testbench used to verify the AES decryption core.

PT Agent, CT Agent, and KT Agent are parameterizations of a simple ready/valid interface agent. Agents drive and observe handshakes on their respective interfaces and publish events to their respective scoreboards. Each agent attaches to an interface which provides driving and monitoring tasks and defines pin-level assertions. The decode and expand scoreboards hook into a software decryption model that is used as a reference for the verification.

Formal Methods

In addition to the functional verification techniques described above, the design will be verified using two formal techniques powered by VC Formal Datapath and the HECTOR prover. In particular, two approaches seem particularly interesting:

1. I hope to prove equivalence between the DUT and an implementation in a higher-level language. Synopsys's HECTOR prover is designed to prove datapath equivalence between RTL and C++ implementations of mathematical functions. By combining this with MATLAB Coder, which can automatically generate C++ code from MATLAB models, I should be able to extend this capability to MATLAB models as well.
2. Using the fact that decryption is the mathematical inverse of encryption, I hope to prove that the core is the logical inverse of a software model of the AES encryption algorithm.

HECTOR contains some additional functionality that should help to ensure that this problem is tractable. By directing the prover to determine equivalence and inverseness of the individual modules in the design (such as InvMixColumns and InvSubBytes) rather than across the entire loop at once, the formal proof becomes much more tractable.

Preliminary Testbench Components

Currently, I have completed the framework for the DUT-wiring portion of the testbench. The code that instantiates the DUT and the relevant interfaces that bind to it have been implemented. The interfaces contain tasks for monitoring and driving DUT pins, as well as assertions that verify the properties labeled as "assert" in the verification plan above. These interfaces will be passed to the agents in the AES environment detailed above.

The code is still in its early phases; the modules and assertions currently implemented likely have several bugs. Additionally, much of the UVM framework is not yet implemented. In particular, the agents, scoreboard, tests, stimulus, and reference model still have to be written. The testbench and interfaces that have been implemented are below. I do not have any preliminary simulation results at this time.

AES Core Test Bench

```
// aes_tb.sv
//
// Top-level testbench for the AES decryption core. Instantiates the DUT,
// binds the various interfaces, places virtual interfaces in the config db,
// and runs the test.
//

module aes_tb ();

   // Overridable key length parameter
   `ifdef P_KEYLEN
      parameter int KEYLEN = `P_KEYLEN;
   `else
      parameter int KEYLEN = 128;
   `endif

   parameter int KSCH_BUFFER_DEPTH =
      (KEYLEN == 128) ? 10 :
      (KEYLEN == 192) ? 12 :
      (KEYLEN == 256) ? 14 : 0;

   // Clock and Reset Instantiation
   wire clk;
   wire rst;

   // Interface Instantiation
   rv_if#(KEYLEN) m_kt;
   rv_if#(128) m_ct;
   rv_if#(128) m_pt;
   ksch_buffer_if#(KSCH_BUFFER_DEPTh) m_ksch_buffer;

   // Parameterizable DUT instantiation
   aes_core_wrapper#(KEYLEN) dut_wrapper (.*);

   // TODO: Run the test and configure the environment through the config db.

endmodule
```

AES Core Adapter

```
// aes_core_adapter.sv
//
// Adapter for the AES core. Bridges the interfaces used by the agents to the
// wire definitions used by teh DUT.
//

module aes_core_wrapper#(
   parameter int KEYLEN
```

```
) (
    input wire clk,
    input wire rst,
    rv_if.slv m_kt,
    rv_if.slv m_ct,
    rv_if.mstr m_pt,
    ksch_buffer_if.dut m_ksch_buffer
);

    // Glue wire instantiation
    wire [0:127] kt;
    wire         kt_vld;
    wire         kt_rdy;

    wire [0:127] ct;
    wire         ct_vld;
    wire         ct_rdy;

    wire [0:127] pt;
    wire         pt_vld;

    // Interface assignment
    assign kt     = kt.data;
    assign kt_vld = kt.valid;
    assign kt.ready = kt_rdy;

    assign ct     = ct.data;
    assign ct_vld = ct.valid;
    assign ct.ready = ct_rdy;

    assign pt     = pt.data;
    assign pt_vld = pt.valid;

    // Parameterizable DUT instantiation
    generate
        if (KEYLEN == 128) begin
            aes_decrypt128 dut (.*);
        end else if (KEYLEN == 192) begin
            aes_decrypt192 dut (.*);
        end else if (KEYLEN == 256) begin
            aes_decrypt256 dut (.*);
        end else begin
            initial begin
                `uvm_fatal("CONFIGURATION", $sformat("Invalid keylength: KEYLEN=%d", KEYLEN))
            end
        end
    endgenerate

endmodule
```

Ready/Valid Interface

```
// rv_if.sv
//
// Interface for a generic ready/valid interface. Bound to the DUT and passed
// to the Decrypter and Expander agents for monitoring and driving.
//
// Implements a pin-level X-Propagation check.
//

interface rv_if#(
  parameter int DATA_W
) (
  inout logic clk,
  inout logic rst
);

  // Signal declarations
  logic            valid;
  logic            ready;
  logic [DATA_W-1:0] data;

  // Modport definitions
  modport mstr (
    input  ready,
    output valid, data
  );

  modport slv (
    input  valid, data,
    output ready
  );

  // Clocking block definitions
  clocking mon_cb @(posedge clk);
    default input #1step;
    input valid, data, ready;
  endclocking

  clocking mstr_cb @(posedge clk);
    default input #1step output #1step;
    input ready;
    output valid, data;
  endclocking

  clocking slv_cb @(posedge clk);
    default input #1step output #1step;
    input valid, data;
    output ready;
  endclocking
```

```
    // Reusable interface monitoring/driving tasks
    task automatic wait_handshake();
        forever begin
            @(mon_cb);
            if (mon_cb.rkey_vld_decode === 1)
                break;
        end
    endtask

    task automatic get_data(inout logic [DATA_W-1:0] data_out);
        wait_handshake();
        data_out = data;
    endtask

    task automatic drive_data(input logic [DATA_W-1:0] data_in);
        mstr_cb.valid <= 1'b1;
        mstr_cb.data  <= data_in;
        wait_handshake();
        mstr_cb.valid <= 1'b0;
        mstr_cb.data  <= 'x;
    endtask

    task automatic drive_ready(input bit ready);
        slv_cb.ready <= ready;
        @(rd_slv_cb);
    endtask

    // Asynchronous ready/valid X-Propagation assertion.
    property p_rv_x_prop;
        @(posedge clk) disable iff (rst)
        valid |-> !$isunknown(data);
    endproperty
    a_p_rv_x_prop: assert property (p_rv_x_prop)
    else $error("Invalid data on ready/valid interface");
    c_p_rv_x_prop: cover property (p_rv_x_prop);

endinterface
```

Key Schedule Buffer Interface

```
// ksch_buffer_if.sv
//
// Interface for the key schedule buffer. Bound to the DUT and passed to the
// Key Schedule Buffer Agent for passive monitoring of the round key.
//
// Implements pin-level assertions including a LIFO ordering check and
// X-Propagation checks.
//

interface ksch_buffer_if#(
```

```
    parameter int KSCH_BUFFER_DEPTH
) (
    inout logic clk,
    inout logic rst
);

    // Decrypt facing round key interfcae.
    logic [127:0] rkey_decrypt;
    logic         rkey_vld;
    logic         next_rkey_decrypt;

    // Expand facing round key interfcae.
    logic [127:0] rkey_keyexp;
    logic [127:0] rkey_vld_keyexp;

    // Modport definitions
    modport dut (
        output rkey_decrypt, rkey_vld_decrypt, next_rkey_decrypt, rkey_keyexp, rkey_vld_keyexp
    );

    modport tb (
        input  rkey_decrypt, rkey_vld_decrypt, next_rkey_decrypt, rkey_keyexp, rkey_vld_keyexp
    );

    // Clocking block definitions
    clocking tb_cb @(posedge clk);
        default input #1step output #1step;
        input  rkey_decrypt, rkey_vld_decrypt, next_rkey_decrypt, rkey_keyexp, rkey_vld_keyexp;
    endclocking

    // Reusable interface monitoring tasks
    task automatic rkey_keyexp_wait_handshake();
        forever begin
            @(tb_cb);
            if (tb_cb.rkey_vld_keyexp === 1)
                break;
        end
    endtask

    task automatic rkey_keyexp_get_key(inout logic [127:0] key);
        rkey_keyexp_wait_handshake();
        key = tb_cb.rkey_keyexp;
    endtask

    task automatic rkey_decode_wait_handshake();
        forever begin
            @(tb_cb);
            if (tb_cb.rkey_vld_decode === 1)
                break;
        end
```

```
endtask

task automatic rkey_decode_get_key(inout logic [127:0] key);
  rkey_decode_wait_handshake();
  key = tb_cb.rkey_decode;
endtask

// Asynchronous ksch buffer data predictor
logic [127:0] expected_data[KSCH_BUFFER_DEPTH];
integer expected_wr_ptr;
integer expected_rd_ptr;
always @(posedge clk) begin
  if (rst) begin
    expected_wr_ptr <= 0;
    expected_rd_ptr <= KSCH_BUFFER_DEPTH-1;
  end else begin
    // Write pointer advance
    if (rkey_vld_keyexp) begin
      expected_data[expected_wr_ptr] <= rkey_decrypt;
      expected_wr_ptr <= expected_wr_ptr + 1 == KSCH_BUFFER_DEPTH ? 0 :  expected_wr_ptr + 1;
    end

    // Read pointer advance
    if (rkey_vld_decode) begin
      expected_rd_ptr <= expected_rd_ptr == 0 ? KSCH_BUFFER_DEPTH - 1 : expected_rd_ptr - 1;
    end
  end
end

// Asynchronous ksch buffer order check
property p_ksch_buffer_lifo;
  @(posedge clk) disable iff (rst)
  rkey_vld_decrypt |-> rkey_decrypt == expected_data[expected_rd_ptr];
endproperty
a_p_ksch_buffer_lifo: assert property (p_ksch_buffer_lifo)
else $error("Data ordering missmatch!");
c_p_ksch_buffer_lifo: cover property (p_ksch_buffer_lifo);

// X-Propagation check for the decode side of the buffer
property p_ksch_buffer_decrypt_x_prop;
  @(posedge clk) disable iff (rst)
  rkey_vld_decrypt |-> !$isunknown(rkey_decrypt);
endproperty
a_p_ksch_buffer_decrypt_x_prop: assert property (p_ksch_buffer_decrypt_x_prop)
else $error("Invalid data on rkey_decrypt");
c_p_ksch_buffer_decrypt_x_prop: cover property (p_ksch_buffer_decrypt_x_prop);

// X-Propagation check for the expander side of the buffer
property p_ksch_buffer_keyexp_x_prop;
  @(posedge clk) disable iff (rst)
```

```
    rkey_vld_keyexp |-> !$isunknown(rkey_keyexp);
  endproperty
  a_p_ksch_buffer_keyexp_x_prop: assert property (p_ksch_buffer_keyexp_x_prop)
  else $error("Invalid data on rkey_keyexp");
  c_p_ksch_buffer_keyexp_x_prop: cover property (p_ksch_buffer_keyexp_x_prop);

endinterface
```

Conclusions

This report defines the verification plan for the AES Decryption Core from OpenCores. Our verification effort is focused on determining functional equivalence with a software reference model of AES decryption through constrained random verification techniques. This is supported by directed testing on known test vectors and experimental formal verification techniques. Through these techniques, we hope to verify both the correctness of our software reference model and the design under test.

Our verification plan identifies two coverage-critical areas: the Sbox and InvSbox modules. These modules implement the S-box and inverse S-box transforms as hard-coded, 256-element lookup tables. Covering every possible input to these lookup tables is critical to verifying the correctness of the design as a whole. Other modules along the datapath implement simpler functions that should break in predictable ways. The risk for these modules is relatively low, so a black-box testing approach combining constrained-random stimulus with directed NIST test vectors at the top level interfaces is sufficient. Targeting 100% coverage on the Sbox and InvSbox modules may seem difficult to achieve, but the pseudo-randomness properties of the AES algorithm should help to ensure that our randomly generated test vectors effectively test the whole design.

Overall, though there is a lot of verification code left to implement, the project is progressing quite well. A basic framework has been laid out for the testbench, and plans are in place for the environment, stimulus, and required tests. One of the most difficult parts that remains is the implementation of the reference model. Even so, I should be able to complete the model and test environment within the required time frame. In the near future, I will continue to develop this test environment to start serious constrained-random testing within two weeks.